



Artificial Intelligence in Pla...

Recursive Language Models: The Architecture That Thinks in Layers



Ms. Adeeba

Follow

7 min read · Mar 9, 2026



How teaching AI to think recursively could change the way machines understand language

Language is not flat. When you say “*The cat that chased the mouse that ate the cheese ran away*” — your brain doesn’t process that word by word in a straight line. It builds structure. It nests meaning inside meaning. It recurses.

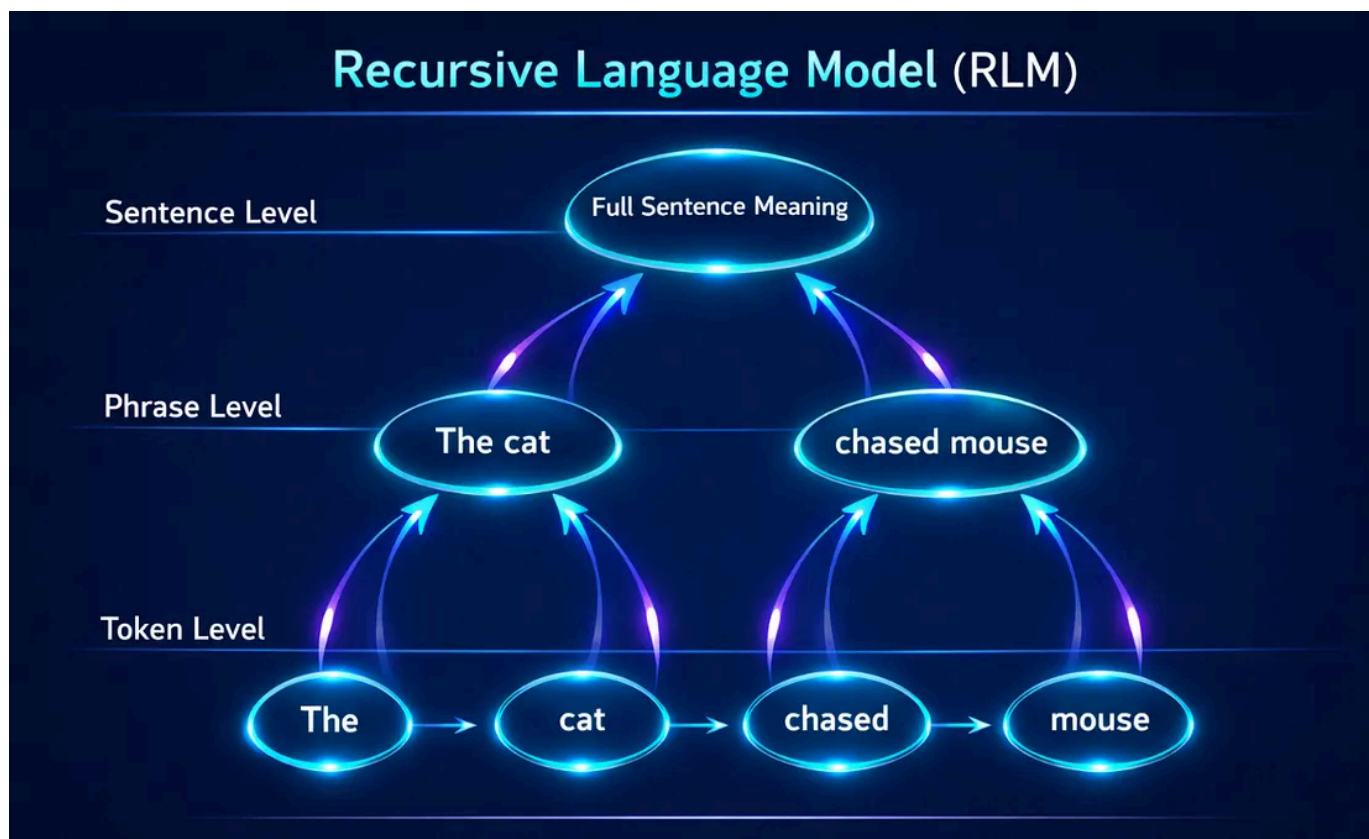
Most modern AI doesn’t do this. It reads left to right, token by token, flattening the beautiful hierarchy of language into a long sequence. But a growing class of models — **Recursive Language Models (RLMs)** — are trying to change that.

What Is a Recursive Language Model?

A Recursive Language Model is any language model that applies itself — or a core operation — **repeatedly across hierarchical levels of structure**, rather than processing text as a flat sequence.

The idea draws directly from how language actually works. Grammar is recursive by nature. A sentence contains clauses. Clauses contain phrases. Phrases contain words. And at any level, you can nest more of the same. Noam Chomsky pointed this out decades ago — human language has **infinite generative capacity** from finite rules, and recursion is the engine behind it.

RLMs try to make AI respect that same structure.



A Brief History: Where Did This Come From?

The story starts with **Richard Socher** at Stanford around 2011–2013. He introduced **Recursive Neural Networks (RecNNs)** — a class of models that

processed sentences by walking up a parse tree. Start at the leaves (individual words), compose them into phrases, compose phrases into clauses, and finally arrive at a single vector representing the entire sentence's meaning.

The same neural network — the same weights — were reused at every node. That's the recursive part. One function, applied at every level of a tree.

It was elegant. It was linguistically motivated. And it worked surprisingly well for tasks like sentiment analysis, where understanding nested negation (*“not entirely without merit”*) actually mattered.

But it had a painful dependency: you needed a **parse tree** to begin with. That meant running a separate parser first, which was slow, brittle, and became a bottleneck.

The field moved on. Transformers arrived and brute-forced the problem with attention mechanisms and scale. But the core insight never went away.

The Core Architectures

1. Recursive Neural Networks (RecNN)

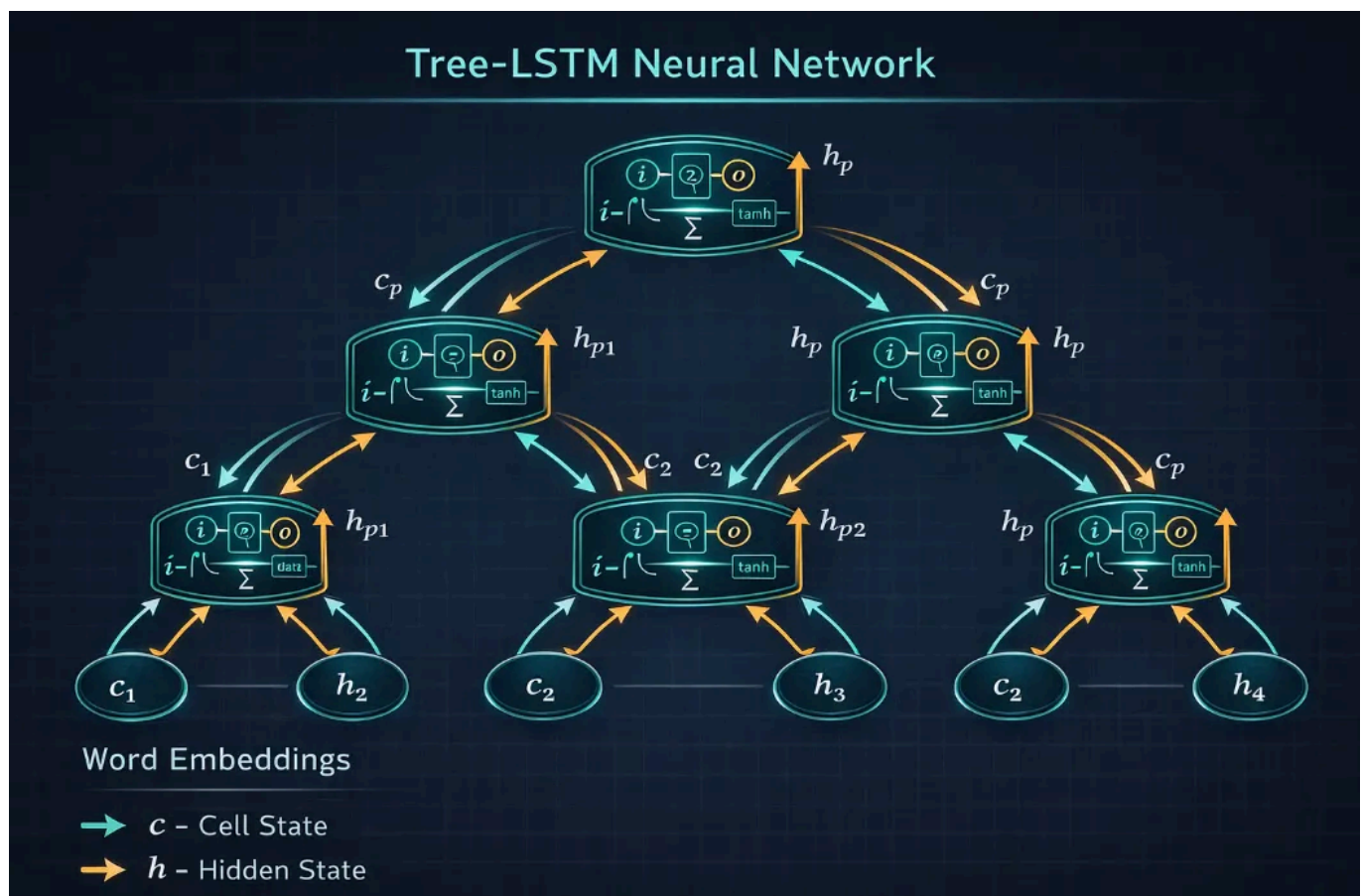
The original. A single weight matrix applied bottom-up through a syntax tree. Powerful for compositional tasks, limited by its dependence on external parsing. Think of it as the theoretical ideal — mathematically clean, practically constrained.

2. Tree-LSTM

An upgrade to the classic LSTM that replaces the linear chain with a **tree structure**. Each node in the tree receives hidden states from its children, not just the previous timestep. Two main variants emerged:

- **Child-Sum Tree-LSTM**: averages children's states — good for unordered children
- **N-ary Tree-LSTM**: treats each child position separately — better for binary parse trees

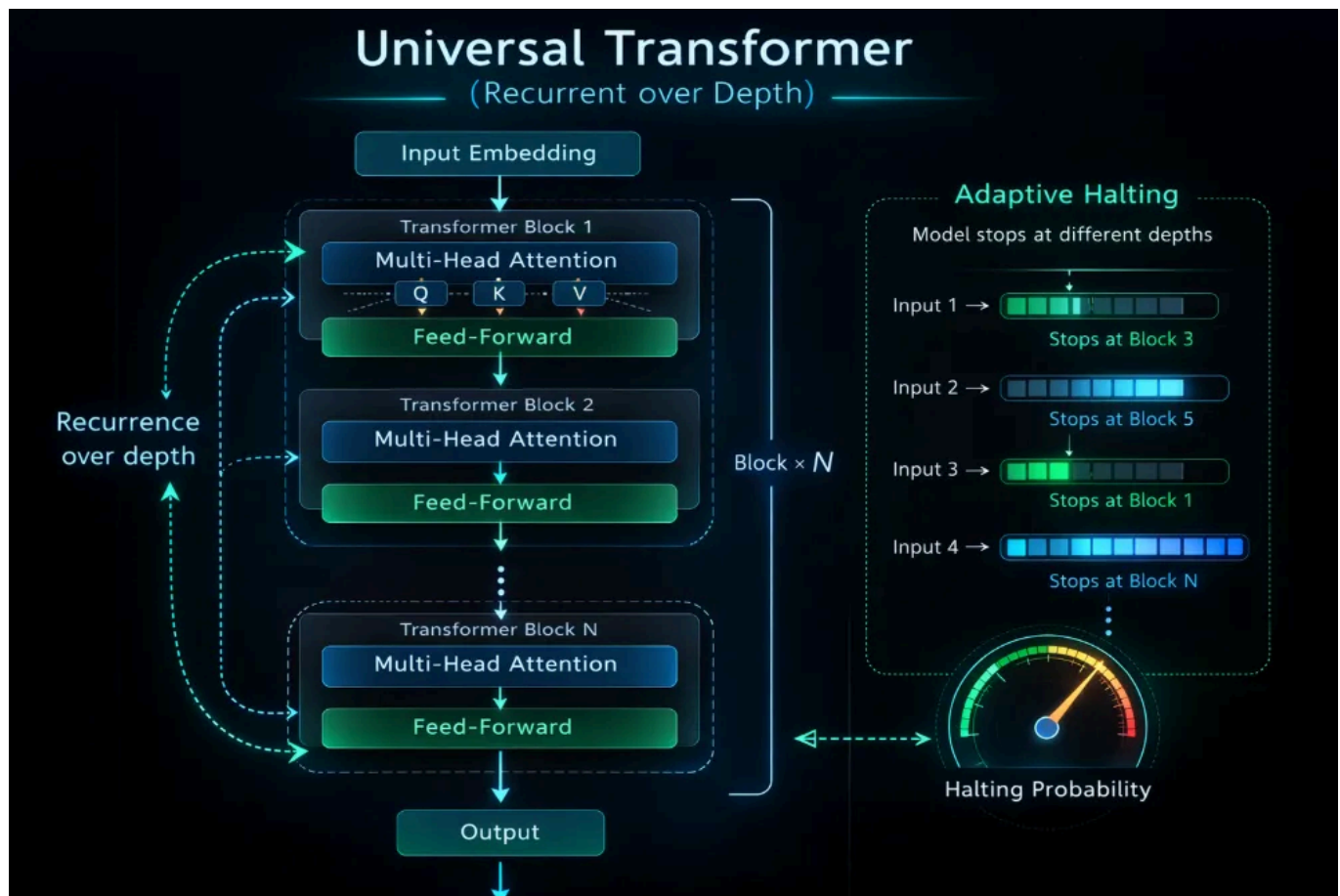
Tree-LSTMs significantly outperformed flat LSTMs on tasks requiring understanding of sentence structure, like semantic relatedness and sentiment classification.



3. Universal Transformer

A pivotal architecture from Google (2018). Instead of stacking *different* layers like a standard transformer, the Universal Transformer applies the **same layer repeatedly** — with a recurrence over depth rather than sequence position.

This gives it properties of both transformers (parallelism, attention) and RNNs (depth-wise recursion). It can theoretically keep refining its representations until it's confident — using an **adaptive computation mechanism** to halt early on simple inputs and run longer on hard ones.



4. Hierarchical Attention Transformers

Models like Longformer, BigBird, and HAT (Hierarchical Attention Transformer) apply attention at multiple granularities:

- Token-level attention within a sentence
- Sentence-level attention within a paragraph
- Paragraph-level attention within a document

Each level builds on the representations from below. It's not strictly recursive in the mathematical sense, but it embodies the same **hierarchical compositionality** principle.

5. Neural Module Networks (NMNs)

A fascinating direction from the visual QA world, now crossing into language. NMNs dynamically assemble a **computation graph of specialized sub-modules** based on the structure of the query. The graph itself can be recursive — modules feed into other modules, forming a tree of operations.

Get Ms. Adeeba's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Ask “*Is the object to the left of the red cube larger than the cylinder?*” and the model builds a graph: [Find red cube] → [Find object to left] → [Find cylinder] → [Compare sizes]. Pure compositional reasoning.

6. Recursive Self-Prompting (LLM-Era Recursion)

The newest form, and arguably the most practically impactful. Rather than recursion in weights or architecture, this is **recursion at the prompt level**:

- **Least-to-Most Prompting:** Decompose a complex problem into sub-problems. Solve the simplest first. Feed results forward. Solve the next level. Repeat.
- **Recursive Summarization:** Split a 100,000-word document into chunks. Summarize each. Summarize the summaries. Recurse until it fits in a context window.
- **Multi-agent orchestration:** An outer LLM breaks a task into subtasks, spins up inner LLM calls to handle each, then synthesizes results. The same model, applied recursively, at different levels of abstraction.

Why Does Recursion Matter?

Here's the core argument for RLMs over flat sequence models:

Language is compositional. The meaning of “*not bad*” is not the sum of “*not*” and “*bad*” processed independently. It's a composed structure where one modifies the other. Flat models learn to approximate this through attention and scale. Recursive models build it in structurally.

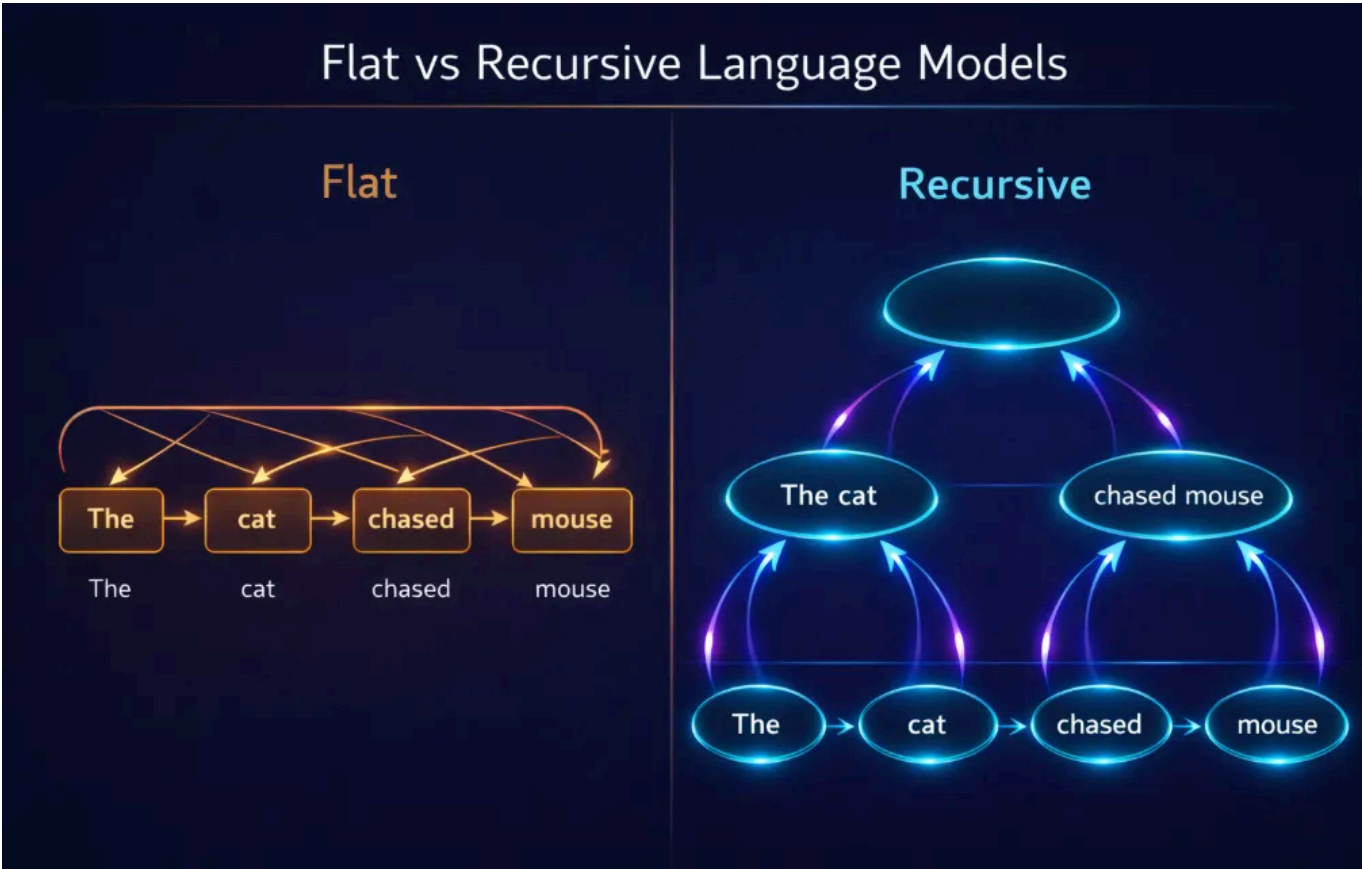
Compositionality generalizes better. A model that understands how parts combine to form wholes can handle combinations it has never seen before. This is called **systematic generalization** — and it's one of the most consistent weaknesses of flat transformers on tasks that require it.

Recursion matches the task structure. Many real-world tasks are inherently hierarchical: legal documents, codebases, scientific papers, multi-step reasoning chains. A model that processes these with matching recursive structure has a natural alignment advantage.

The Trade-offs

Recursive architectures aren't free. Here's an honest look at the costs:

Property	Flat Transformers	Recursive Models
Scalability	Excellent	Challenging
Parse dependency	None	Often required
Compositionality	Learned implicitly	Built in structurally
Training stability	High	More complex
Interpretability	Low	Higher
Current SOTA	Yes	Competitive on specific



The honest truth is that **scale has been winning**. GPT-4, Claude, Gemini — these are all flat transformers trained on enormous data, and they've

achieved remarkable results without explicit recursion. They've learned to approximate hierarchical reasoning through sheer scale.

But approximation has limits. As we push into tasks requiring **deep compositional reasoning** — formal logic, complex code generation, multi-hop question answering — the cracks in flat models become visible.

The Renaissance: Recursion Is Coming Back

Several recent trends suggest RLM ideas are having a quiet resurgence:

Chain-of-Thought Reasoning is essentially recursive decomposition at inference time. You're asking the model to build a hierarchy of reasoning steps before arriving at an answer.

Multi-agent systems (like AutoGPT, LangGraph, CrewAI) implement recursion at the system level — orchestrator models calling specialist models, which may themselves call other models.

Mixture of Experts (MoE) architectures have a recursive flavor — routing inputs through different sub-networks based on content, composing specialized knowledge.

Neurosymbolic AI research is explicitly trying to combine the structural guarantees of symbolic recursion with the learning power of neural networks. This is arguably where RLMs find their most rigorous future home.

What's Next?

The most exciting frontier is **learned structure** — models that don't rely on external parse trees but instead **induce their own hierarchical structure** during training. Models like the **Ordered Neurons LSTM** (ON-LSTM) and various **latent tree models** try to do exactly this: learn to parse and compose simultaneously, in an end-to-end differentiable way.

If that problem gets solved cleanly, you get the best of both worlds: the scalability of modern deep learning with the structural elegance of recursive composition.

The flat-vs-recursive debate in NLP isn't over. It's just moved from architecture diagrams into prompt engineering, multi-agent systems, and the quiet research labs still convinced that **structure matters**.

They might be right.

Key Takeaways

- Recursive Language Models process text hierarchically, matching the nested structure of natural language
- The core idea dates to Socher's RecNNs (2011) and has evolved through Tree-LSTMs, Universal Transformers, and now LLM-era self-prompting
- Recursion is architecturally motivated by **compositionality** — the principle that meaning is built from parts
- Modern LLMs approximate recursive reasoning through scale; RLMs build it in structurally
- The field is experiencing a revival through chain-of-thought, multi-agent systems, and neurosymbolic research